Heat transport in chaotic magnetic fields

Mireia Raventós Amadó, mireia.raventosamado@epfl.ch
École Polytechnique Fédérale de Lausanne (EPFL), Swiss Plasma Center (SPC)

Supervisor: Joaquim Loizu, joaquim.loizu@epfl.ch

Autumn semester 2026

1 PyOculus workflow

In `PyOculus`, we can define a magnetic configuration by specifying the components of the magnetic field **B**. In our case, we want to study the following magnetic field in slab geometry, in which a perturbation creates an island structure:

$$\begin{cases} B_x = m\epsilon(x)\sin\alpha \\ B_y = \iota'(x - x_0) + \epsilon'(x)\cos\alpha \\ B_z = 1 \end{cases} \tag{1}$$

For a system with $x \in [0, 1]$ containing an island at $x_s = 0.5$, as a simple model for a perturbation localized around the rational surface, we choose

$$\epsilon(x) = \delta \cdot x(1 - x)\,. \tag{2}$$

2 Modifications on PyOculus

In order to reproduce the magnetic configurations in slab geometry I worked with on TPIV2, I had to introduce some modifications on the PyOculus library.

2.1 Magnetic field in the slab model

I created two new classes:

- `SlabBfieldOnePert()`: Stemming from the `ToroidalBfield()` class, this class implements a magnetic field containing one magnetic island in slab geometry.
- `SlabBfieldTwoPerts()`: Stemming from the `ToroidalBfield()` class, this class implements a magnetic field containing two magnetic islands in slab geometry.

These changes haven't been merged into the package, for now they are implemented in files located in the `SdS/local/pyoculus_mod` folder.

2.2 Change of the domain of the toroidal map

In order to generate the map corresponding to a magnetic field in slab geometry, I use the class `ToroidalBfieldSection()`. This class had the domain of the x coordinate set to $[-1, 1]$. Since in some cases I want to work with a different domain, I introuced a small modification in the definition of the class, so that a user can specify the domain of x when creating an instance of the class, while preserving $[-1, 1]$ as the default domain.

This change has been made in the forked version of the library, in the file `pyoculus/pyoculus/maps/toroida` lines 18-19, and committed and pushed in the branch `slab-model`.

2.3 Untranspose Jacobian matrix

The method `ToroidalBfieldSection.df()` was returning the transposed of the Jacobian matrix

$$J_f(x) = \begin{bmatrix} \partial_{x^1} f^1 & \cdots & \partial_{x^n} f^1 \\ \vdots & \ddots & \vdots \\ \partial_{x^1} f^n & \cdots & \partial_{x^n} f^n \end{bmatrix} . \tag{3}$$

This caused some problems when calculating the eigenvectors of the manifolds, so I modified the method so that it would return $J_f(x)$ instead of $J_f^T(x)$.

3 Various notes

Use `find_clinics_exhaustive`() - mirar una mica més ne detall què fa exactament, però en prinicipi hauria de trobar totes les intereccions que necessitem, no només una com fa `find_clinic_single`().

Com que estem treballant amb heteroclinics, hem de donar 2 fp. Donar un O-point ajuda al programa.

Per calcular els manifolds, utilitzar `compute_adaptive`() (pull from Ralf) - comprovar resolució.

3.1 Coses a fer

- Mirar problemes amb find_clinic_exhaustive().
- Revisar jacobià de coordenades cilíndriques: buscar terme 1/r al integrator (al slab model no hauríem de tenir-ne) $\implies$ No hi ha cap terme degut a que la llibreria agafa el camp magnètic en forma contravariant: les correccions geomètriques estan integrades en el tensor mètric i els símbols de Christoffel (i.e, en els operadors geomètrics).
- FP en el caos: buscar $t > 1$ (relació entre t i n,m?)